



POLITECNICO
MILANO 1863



ITECNICO
LANO 1863

architettura dei calcolatori e sistemi operativi

assembler – introduzione
RISC V – capitolo 2 P&H

assembler RISC V (**RV64i** – aritmetica intera a 64 bit)

libro di testo Patterson & Hennessy (P&H)

- **capitolo 2** (esclusi riferimenti a Java)
<https://www.zanichelli.it/ricerca/prodotti/struttura-e-progetto-dei-calcolatori-edizione-risc>

strumenti di supporto

- RARS – simulatore a livello ISA
 - *è lo strumento più facile e immediato da capire e usare*
 - richiede solo di installare il supporto per linguaggio Java
 - download: <https://github.com/TheThirdOne/rars>
- WebRISC-V – simulatore a livello micro-architetturale
 - modella e simula la pipeline RISC V (quasi come in P&H)
 - semplice e utile per *esplorare* le funzioni della pipeline
 - si usa via navigatore Web: <https://web riscv.dii.unisi.it/>
- Altri simulatori a livello ISA e/o micro-architetturale:
 - EMULSIV – simula la ISA e (in parte) la pipeline RISC V:
<http://tice.sea.eseo.fr/riscv>
 - RIPES – simile a WebRISC-V (ma va installato in locale)
- Si veda anche su <https://riscv.org/> per info e annunci

RV

64 == <num. bit che
compongono un
indirizzo di memoria/
num. bit in registri>

i == 32 registri general
purpose,

e == 16 reg.,

*A seguire lettere che
indicano gli insiemi
estensione (M,A,F,D,...)*

*Le istruzioni sono
sempre codificate con
32 bit, tranne per
l'estensione **c**: che
prevede anche istr.
codificate con 16 bit)*



perché studiare il linguaggio assembler

contro

- è noioso ... **ABBASTANZA**
- è difficile **FALSO**
- non serve a niente **FALSO e INGANNEVOLE**

pro

- aiuta a capire come funziona veramente un calcolatore
- aiuta a scrivere meglio programmi in linguaggio ad alto livello, perché si capisce come vengono eseguiti
- è il più potente linguaggio di programmazione:
 - ✓ consente l'accesso a tutte le risorse del computer
 - ✓ permette di ottimizzare al massimo il programma



valore pedagogico

imparare il linguaggio macchina / assembler vi aiuterà a capire

- le ripercussioni delle scelte di architettura del sistema
- il funzionamento e la realizzazione dei servizi offerti dai linguaggi ad alto livello
- quali sono le ottimizzazioni che il compilatore può fare
- e quelle che può fare il programmatore, anche in linguaggi ad alto livello, e quindi aumentare l'efficienza del codice

anche se non scriverete mai un intero programma in linguaggio assembler, vi capiterà di dovere accedere a caratteristiche di basso livello della macchina

- approccio ibrido: potete usare il linguaggio macchina esattamente nei punti critici dove serve, e usare il vostro linguaggio di alto livello preferito nel resto del progetto
- questo avviene spesso nella programmazione di sistemi embedded, per il controllo dei processi con vincoli di tempo reale (real-time)



livelli di astrazione

Programma
in linguaggio
ad alto livello
(in C)

```
scambia(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compilatore

Programma
in linguaggio
assembler
(per RISC-V)

```
scambia:
    slli x6, x11, 3
    add  x6, x10, x6
    ld   x5, 0(x6)
    ld   x7, 8(x6)
    sd   x7, 0(x6)
    sd   x5, 8(x6)
    jalr x0, 0(x1)
```

Assemblatore

Programma
in linguaggio
macchina binario
(per RISC-V)

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000001000000001100111
```

scendendo di livello, diventiamo
meno astratti e più concreti

- il livello astratto omette dettagli,
ma permette di trattare la *complessità*

linguaggio ad alto livello

- produttività e portabilità del codice

linguaggio assembler

- rappresentazione testuale delle istruzioni

linguaggio macchina

- codifica di istruzioni e dati in binario

Quali sono i dettagli che via via scopriamo
scendendo di livello ?



livelli di astrazione

il **linguaggio di alto livello** “astrae” dalla specifica piattaforma, fornendo costrutti più semplici e generali

- è una “interfaccia generica”, valida per qualunque computer (con tutti i vantaggi che ciò comporta)
- ma proprio perché è uguale per tutte le macchine, NON può fornire accesso alle funzionalità specifiche di una determinata macchina

il **linguaggio macchina / assembler** permette di fare cose fisicamente impossibili per altri linguaggi (come accedere e manipolare schede grafiche, registri del processore, ecc.)

il **linguaggio assembler** è a più alto livello del **linguaggio macchina**

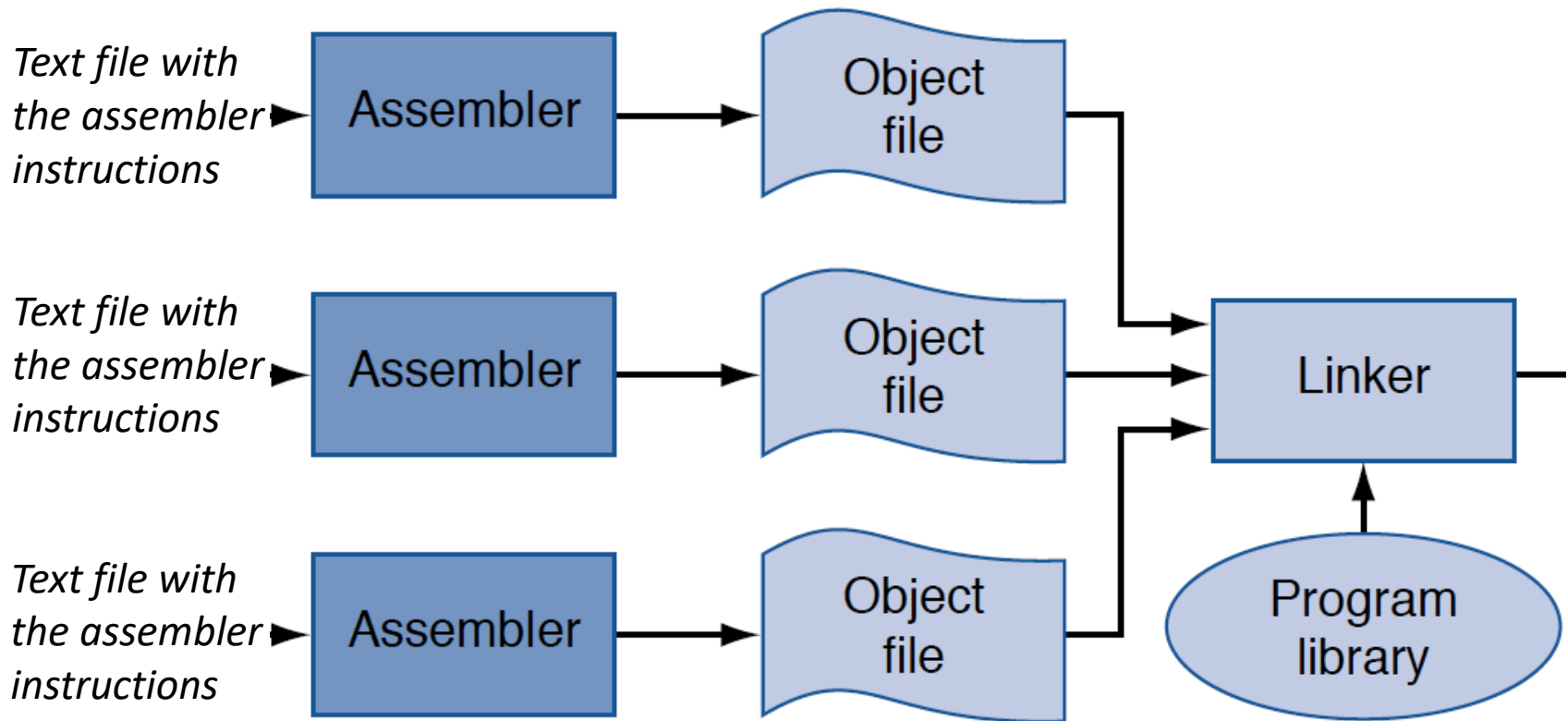
- rappresentazione alfanumerica (mnemonica) delle istruzioni
- consente al programmatore di ignorare il formato binario del linguaggio macchina



processo di compilazione / esecuzione

Il passaggio dal codice sorgente all'esecuzione del programma si articola in tre fasi:

1. compilazione
2. collegamento (o *linking*)
3. caricamento ed esecuzione



linguaggio assembler

linguaggio **specifico** del particolare processore

più **primitivo** rispetto ai linguaggi ad alto livello

- p. es., il controllo del flusso è poco sofisticato
(non ci sono cicli *for* o *while*, né condizionali *if-then-else*)

linguaggio molto **restrittivo**

- p. es., le istruzioni aritmetiche hanno un numero piccolo e fisso di operandi

studieremo la **ISA** (Instruction Set Architecture) di RISC V

- simile ad altre architetture sviluppate a partire dagli anni '80

due processori con lo stesso linguaggio macchina hanno la stessa architettura, anche se le realizzazioni hardware possono differire

scopi di progetto della ISA: massimizzare le prestazioni (per es.: num. di istruzioni eseguite al secondo) – minimizzare i costi, anche riducendo i tempi di progetto



concetto di *stored program*

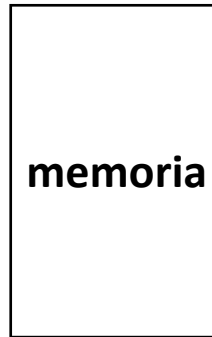
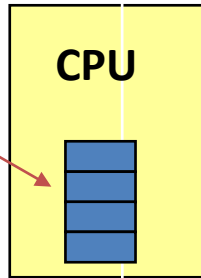
i programmi sono **sequenze di istruzioni macchina** rappresentate in binario

- stringhe di bit con un dato formato di rappresentazione

i **programmi** (come i **dati**) sono **codificati in binario e caricati in memoria** per l'esecuzione

- la CPU (processore) legge le istruzioni dalla memoria (come i dati)

banco di registri
(*register file*)



memoria per dati e programmi
(*espressi in linguaggio macchina*)

ciclo di **prelievo ed esecuzione** (*fetch & execute*)

- CPU **legge** (o **preleva**) (*fetch*) l'istruzione corrente (indirizzata dal *Program Counter*), e la pone in un registro speciale interno (*Instruction Register*)
- CPU usa i bit dell'istruzione per *controllare* le azioni da svolgere, e su questa base **esegue** l'istruzione
- CPU determina la *prossima* istruzione da eseguire e ripete ciclo



stored program

la CPU vede tutto come codici numerici (cioè stringhe di bit), che rappresentano

- dati: interi, caratteri e numeri in virgola mobile (floating point)
- istruzioni

un programma (istruzioni + dati) è una sequenza di codici numerici binari

la CPU interpreta alcuni codici numerici come istruzioni

```
001001111011110111111111111100000
101011111011111100000000000010100
10101111101001000000000000100000
10101111101001010000000000100100
1010111110100000000000000011000
1010111110100000000000000011100
1000111110101110000000000011100
1000111110111000000000000011000
0000000111001110000000000011001
0010010111001000000000000000001
0010100100000001000000001100101
1010111110101000000000000011100
00000000000000000111100000010010
00000011000011111100100000100001
0001010000100000111111111110111
10101111101110010000000000011000
00111100000001000001000000000000
10001111101001010000000000011000
00001100000100000000000011101100
00100100100001000000010000110000
10001111101111110000000000010100
00100111101111010000000000100000
0000001111100000000000000001000
00000000000000000001000000100001
```



assembler – linguaggio per umani

- leggere numeri non è molto facile
- per poterlo leggere conviene tradurre questi codici numerici in stringhe alfanumeriche più leggibili $\Rightarrow$ linguaggio assembler
- assembler è un linguaggio di programmazione che rispecchia fedelmente (cioè 1-a-1) le istruzioni del linguaggio macchina
 - uso: più leggibile e flessibile
 - ha bisogno di un traduttore (assemblatore) per convertire il codice in linguaggio macchina
 - CPU con insiemi (o repertori) di istruzioni (*instruction set*) diversi hanno linguaggi assembler diversi



assembler – 1

usando un traduttore inverso (disassemblatore) posso ricavare dalle istruzioni macchina il programma simbolico corrispondente

più semplice da leggere, ma ancora difficile, perché i nomi dei registri e le locazioni di memoria sono espressi tramite indirizzi numerici

```
addi    2, 2, -64
sd       31, 20(2)
sd       5, 32(2)
sd       6, 40(2)
sd       0, 48(2)
sd       0, 56(2)
ld       14, 56(2)
ld       24, 48(2)
sub      14, 6, 5
addi     8, 14, 1
.....
jal      1, 1048812
.....
...
```



assembler – 2

la routine è scritta in linguaggio assembler con le etichette mnemoniche per le varie locazioni di memoria

i comandi che iniziano con il punto sono direttive all'assemblatore

- **.text** o **.data**
 - indica che le linee successive contengono istruzioni o dati
- **.align n**
 - indica l'allineamento su 2^n byte
- **.globl main**
 - l'etichetta *main* è globale, ossia visibile dal codice in altri file
- **.asciiz**
 - area di memoria che memorizza una stringa terminata da NULL

```
.text
.align 3
.global    main
main:
    addi    sp,  sp, -64
    sd      ra,  20(sp)
    sd      a0,  32(sp)
    sd      0,   48(sp)
    sd      0,   56(sp)
loop:
    ld      a4,  56(sp)
    .....
    .....
    jal     printf
    ...
```



assembler – 3

ma qual è il corrispondente programma scritto in C ?

```
#include <stdio.h>

int
main (int argc, char *argv[])
{
    int i;
    int sum = 0;

    for (i = 0; i <= 100; i = i + 1) sum = sum + i * i;
    printf ("The sum from 0 .. 100 is %d\n", sum);
}
```

Il programma assembler RISC-V precedente è un estratto della traduzione del programma C (secondo il compilatore GCC). Le variabili locali *i* e *sum* sono memorizzate in pila. L'area di attivazione (8 parole da 64 bit) di *main* in pila le contiene entrambe, con anche il registro di rientro *ra* salvato. La traduzione del ciclo *for* non è riportata, ma si vede la chiamata alla funzione *printf*. Altri aspetti si chiariranno in seguito.



caratteristiche di un'architettura del set di istruzioni

- registri
- istruzioni macchina e tipi di formati
 - codifica (formato) delle istruzioni
 - tipo e dimensione
 - posizione degli operandi
 - in memoria e/o nei registri interni al processore
 - tipi di dati e dimensione
 - operazioni consentite
 - modalità di indirizzamento
- modello di memoria



linguaggio assembler

le istruzioni comprese nel linguaggio macchina di qualunque computer sono classificabili in quattro categorie:

- aritmetico-logiche (*arithmetic / logic*)
- trasferimento da / verso memoria (*load / store*)
- salto condizionato (*branch*) e incondizionato (*jump*)
per il controllo del flusso di programma
- trasferimento in ingresso / uscita (I/O)



registri

nelle istruzioni assembler di RISC V

operandi  registri

non confondere variabili e registri:

- le variabili sono (potenzialmente) in numero infinito
- i registri sono in numero finito

RISC V

ha 32 registri di uso generale:

- sono numerati da 0 a 31, e indicati come *x0-x31*
- bastano cinque bit per codificarli ($2^5 = 32$)
- hanno anche nomi simbolici (convenzionali)
- il registro *x0* contiene sempre il valore 0 (forzato via HW)
- hanno lunghezza 64 bit (una **parola doppia** – ***double word***)
- possono contenere solo valori interi
 - i valori decimali vanno gestiti tramite istruzioni dedicate e registri in virgola mobile (*floating-point*): *f0, f1, ...* (vedi doc RISC V)



registri

per convenzione si possono usare nomi simbolici per denotare i registri, ad esempio:

s0, s1, . . . per contenere variabili (tipicamente locali/automatiche) in C

t0, t1, . . . per contenere valori temporanei (durante il calcolo delle espressioni)



registri referenziabili



registro	nome ABI	utilizzo
x0	zero	costante 0 – imm modificabile (anche se viene scritta non cambia valore)
x1	ra	return address – indirizzo di rientro da funzione
x2	sp	stack pointer – puntatore alla cima della pila
x3	gp	global pointer – puntatore all'area dati globale QUI NON USATO
x4	tp	thread pointer – puntatore a thread QUI NON USATO
x5 – x7	t0 – t2	valori temporanei (indirizzi, variabili globali e calcolo delle espressioni)
x8	s0 / fp	variabile locale di funzione / frame pointer (se in uso)
x9	s1	variabile locale di funzione
x10 – x11	a0 – a1	valore restituito da funzione (di solito basta a0 – se serve si aggiunge a1)
x12 – x17	a2 – a7	argomenti in ingresso a funzione (max 6 argomenti, gli eccedenti in pila)
x18 – x27	s2 – s11	variabili locali di funzione (max 10 o 11 variabili, le eccedenti in pila)
x28 – x31	t3 – t6	valori temporanei (indirizzi, variabili globali e calcolo delle espressioni)

non referenziabili (*cioè non possono apparire in una istruzione assembler*)

Registri non referenziabili

pc	Program Counter
hi, lo	Registri per moltiplicazioni e divisioni NON CONSIDERATI



formato delle istruzioni RISC V – i

tutte le istruzioni RISC V (nella ISA RV64i) hanno la **stessa dimensione (32 bit = 1 parola)**

i 32 bit hanno un significato diverso a seconda del formato (o tipo) di istruzione

- il tipo di istruzione è riconosciuto in base al valore di alcuni bit (**7 bit**) meno significativi (**codice operativo – OPCODE**)

31	7	6	0
immediato (imm) / indirizzi di registro (rd, rs1, rs2) codici operativi ausiliari (funct)			codice operativo (opcode)

le istruzioni possono

- referenziare (cioè «indicare l'utilizzo di») al **massimo tre registri**: due registri sorgente (*rs1* e *rs2*) e un registro destinazione (*rd*); **i registri sono a 64 bit**
- indicare direttamente un valore numerico (*immediato*); l'immediato ha un significato diverso a seconda del tipo di istruzione

modello di memoria

- gli indirizzi sono a 64 bit, espressi con indirizzamento a byte ... segue che una CPU RISC-V può far uso di una memoria RAM con capienza al più 2^{64}
- le istruzioni hanno indirizzi **allineati distanti quattro unità** (multipli di 4); cioè considerando due istruzioni consecutive in memoria, la seconda istruzione avrà l'indirizzo del 1mo byte a distanza di 4 unità dal primo byte dell'istruzione precedente



formato delle istruzioni RISC V – ii

le istruzioni RISC V sono di **sei** tipi (formati)

formato	istruzioni	immediato	immediato esteso (64 bit)
tipo R (register)	aritmetico-logiche	na	n.a.
tipo I (immediate)	di caricamento da memoria (load) aritmetico- logiche con immediati salto incondizionato & link a registro	12 bit in CPL2	estensione segno
tipo S (store)	di memorizzazione in memoria (store)	12 bit in CPL2	estensione segno
tipo B (branch) (SB in PH)	branch condizionale	12 bit in CPL2, multiplo di due	estensione segno
tipo U (load upper integer)	istruzione lui	20 bit in CPL2	n.a.
tipo J (jump & link) (UJ in PH)	salto incondizionato & link	20 bit in CPL2, multiplo di due	estensione segno

Un valore *immediate*:

- in *load* e *store* è un *offset* rispetto a un registro base per individuare l'indirizzo di memoria riferito al dato
- in *branch* e *jump* è un *offset* rispetto al Program Counter, o altro registro, per individuare l'indirizzo di memoria dell'istruzione destinazione del salto



istruzioni aritmetico-logiche – formato R

in RISC V, un'istruzione aritmetico-logica possiede *tre* operandi: i due registri contenenti i valori da elaborare (*registri sorgente*) e il registro contenente il risultato (*registro destinazione*)

l'ordine degli operandi è **fisso**: prima il registro contenente il risultato dell'operazione e poi i due operandi

l'istruzione assembler contiene il codice operativo e tre campi relativi ai tre operandi

OPCODE DEST, SORG1, SORG2



esempio – istruzione add

add addiziona il contenuto di due registri sorgente *rs1* e *rs2* (*addendi* od *operandi*):

add rd, rs1, rs2

e mette nel registro destinazione *rd* il risultato (*somma*) dell'addizione dei contenuti di *rs1* e *rs2*:

add rd, rs1, rs2 # rd ← rs1 + rs2

esempio: $a = b + c$

add s0, s1, s2

nella traduzione da linguaggio ad alto livello a linguaggio assembler, è il compilatore che associa le variabili (p. es. locali) ai registri



esempio – istruzione sub

sub sottrae il contenuto di due registri sorgente *rs1* e *rs2* (*rs1* è il *minuendo* e *rs2* è il *sottraendo*):

sub rd, rs1, rs2

e mette nel registro destinazione *rd* il risultato (*differenza*) della sottrazione dei contenuti di *rs1* e *rs2*:

sub rd, rs1, rs2 # rd ← rs1 – rs2



istruzioni aritmetico-logiche

il fatto che ogni istruzione aritmetica abbia tre operandi, sempre nella stessa posizione, consente di semplificare lo HW, ma complica alcuni aspetti ...

codice C: **A = B + C + D**
 E = F - A

codice RISC V: **add t0, s1, s2**
 add s0, t0, s3
 sub s4, s5, s0



esempio

$$d = (b + c) - (d - f) + g$$

per eseguire le operazioni di addizione e sottrazione è necessario che le variabili (tipicamente locali) siano nei registri

- **s1** è associato a **b**, **s2** a **c**, **s3** a **d**, **s4** a **f**, **s5** a **g**, e il risultato finale viene messo in **s3** (riutilizzato)
- **t0** e **t1** contengono valori temporanei

```
add t0, s1, s2
```

```
sub t1, s3, s4
```

```
sub t0, t0, t1
```

```
add s3, t0, s5
```



trasferimento con la memoria

sono istruzioni che si occupano di trasferire dati dalla memoria fisica ai registri (*load*) e viceversa (*store*)

formato: **OPERATORE** *registro* indirizzo memoria

operatore:

ld *load double*

sd *store double*



istruzione *load*

l'istruzione di ***load*** trasferisce una copia della parola contenuta in una specifica **locazione** di memoria (a 64 bit) mettendola in un registro della *CPU*, e lasciando inalterata la parola di memoria:

ld rd, *LOC* # rd ← memoria[*LOC*]

- la *CPU* invia l'indirizzo della locazione desiderata alla memoria e richiede un'operazione di «lettura del contenuto della memoria» (cioè «una copia del contenuto della memoria»)
- la memoria effettua la lettura della parola memorizzata all'indirizzo specificato e la trasferisce alla *CPU*



istruzione *store*

l'istruzione di ***store*** trasferisce una parola da un registro della *CPU* (64 bit) mettendola in una specifica **locazione** di memoria, sovrascrivendo il contenuto precedente di quella locazione:

sd rs2, *LOC* # memoria[*LOC*] ← rs2

- la *CPU* invia l'indirizzo della locazione desiderata alla memoria, insieme a una copia della parola che vi deve essere scritta, e richiede un'operazione di scrittura
- la memoria effettua la scrittura della parola all'indirizzo specificato



istruzione *ld*

in RISC V, l'istruzione **ld** ha tre argomenti:

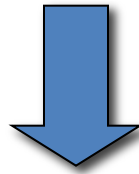
- il *registro destinazione* (*rd*) in cui caricare la parola doppia letta dalla memoria
- una costante o *spiazzamento* (*offset* su 12 bit), in complemento a 2
- un registro base (*base register*) che contiene il valore dell'indirizzo base (*base address*) da aggiungere alla costante; è il registro sorgente numero 1 (*rs1*)

l'indirizzo della parola di memoria da caricare nel registro destinazione è ottenuto sommando la costante estesa in segno (fino a 64 bit) e il contenuto del registro base



istruzione *ld* – trasferimento da memoria a registro

ld s1, 100(s2) # s1 ← memoria[s2 + 100]



al registro destinazione s1 è assegnato il valore contenuto all'indirizzo di memoria contenuto in s2 sommato a 100



base register
*che contiene il valore
di base address*



offset

istruzione *sd* – trasferimento da registro a memoria

possiede argomenti analoghi a quelli di **ld**
i due registri sono le due sorgenti

esempio:

```
sd s1, 100(s2)      # memoria[s2 + 100] ← s1
```

*alla locazione di memoria di indirizzo $s2 + 100$
è assegnato il valore contenuto nel registro $s1$*



esempio – i

ld t0, NUM(s0)

carica in *t0* la parola doppia alla cella di memoria di indirizzo *NUM* + *s0*

sd t0, NUM(s0)

salva alla cella di memoria di indirizzo *NUM* + *s0* la parola doppia in *t0*

dove **NUM** è un'*etichetta simbolica* per indicare il valore di uno spiazzamento (*offset*) che deve essere codificabile **su 12 bit in CPL2** (spesso dichiarata tramite direttiva **.eqv**)

pseudo-istruzioni di caricamento di costante o di indirizzo

li t0, 4 # carica la costante **4** in *t0* (*load immediate*)

la a0, LABEL # carica l'indirizzo (*load address*) **LABEL** in *a0*

dove **LABEL** è un'*etichetta simbolica* per indicare il valore dell'indirizzo di memoria di un dato

... Essendo le pseudo-istruzioni sostituite o espresse dall'assemblatore in una o più istr. native, le costanti/immediati, in questo caso, possono assumere valori a 32 bit in CPL2



esempio – ii

memorizzare (copiare) una parola dalla cella di memoria di indirizzo $NUM1 + s0$ alla cella di indirizzo $NUM2 + s0$, lasciando in $t0$ una copia della parola

```
.eqv NUM1, 100      # come "#define NUM1 100" in C  
.eqv NUM1, 200      # come "#define NUM2 100" in C
```

```
ld t0, NUM1(s0)
```

```
sd t0, NUM2(s0)
```

Nota: la direttiva **.eqv** associa il valore decimale 100 al simbolo NUM1 (similmente per 200 e NUM2). Attenzione: **.eqv** non è un'istruzione macchina, è solo una dichiarazione di costante (come **#define** in C) fatta al traduttore.



trasferimento dati

- la memoria fisica è indirizzata a byte
- può essere vista come un vettore: *memoria [indirizzo]*
- le istruzioni aritmetiche richiedono che gli operandi siano memorizzati nei registri (ciascuno da 8 byte ossia da 64 bit)

esempio

ld s1, offset(s2)

carica in *s1* i 64 bit (parola doppia – double word) all'indirizzo di memoria *s2 + offset*

$s1 \leftarrow \text{memoria}[s2 + \text{offset}]$

sd s1, offset(s2)

salva la parola doppia in *s1* all'indirizzo *s2 + offset*

$\text{memoria}[s2 + \text{offset}] \leftarrow s1$

ci sono anche istruzioni per caricare / memorizzare:

word (32 bit),

half word (16 bit)

e singoli *byte*



load address (la) e load double (ld) – differenze

si presti bene attenzione alla differenza (NOTEVOLE !) tra **la** e **ld** (o **lw**)

Assumiamo sia data una parola doppia di memoria (double word) con contenuto (64 bit):

0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0001
(in esadecimale sono 16 cifre hex: **0x 0000 0000 0000 0001**)

e si supponga che l'indirizzo (pure da 64 bit) di tale parola sia:

0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0010 0011 0100
(in esadecimale sono 16 cifre hex: **0x 0000 0000 0000 00234**)

indicato dal simbolo NUM indichi l'indirizzo (l'associazione tra simbolo NUM e indirizzo numerico si costruisce con una direttiva all'assemblatore che si vedrà in seguito), allora:

la t0, NUM # carica 0x **0000 0000 0000 0234** in t0

ld t0, 0x 234(s1) # carica 0x **0000 0000 0000 0001** in t0

nota: si suppone che il registro **s1** valga zero, dunque si ha $0x\ 234 + s1 = 0x\ 234$

la si riferisce all'indirizzo NUM della parola, **ld** si riferisce al valore della parola doppia (64 bit) di indirizzo NUM (le varianti **lw**, **lh** e **lb** si comportano come **ld**, ma caricano meno bit: 32, 16 o 8)

precisazioni su **la** e **ld** seguiranno: per **la**, quanti bit di NUM vengono caricati (solo i 32 bit meno significativi, mentre i 32 più significativi sono una *estensione di segno*, che nell'esempio vale 0); per **ld**, quanti bit ha lo spiazzamento di *load* (12 bit)



esercizio

leggere il contenuto del **quarto** elemento dell'array A di “parole doppie” e memorizzarlo nel registro **s1**

indirizzo di inizio
dell'array A
nel registro s2

parola doppia [0]
di 64 bit (8 byte)

ld s1, 24(s2)

s2 contiene l'indirizzo del
primo byte in memoria

parola doppia [3]
di 64 bit (8 byte)

[indice] del byte nell'array	cella di memoria (un byte)
0	
1	
....	
7	
8	
....	
....	
22	
23	
24	
25	
....	
31	



come sono memorizzati i valori nella memoria fisica ? in quale ordine ?

una **parola** (doppia) contiene **più byte**:

- p. es., 32 bit ossia 4 byte (64 bit ossia 8 byte)

Le posizioni dei byte di una parola con significato leggibile da sinistra a destra **possono essere enumerate da sinistra a destra o viceversa**:

- **big endian**, le posizioni dei byte vanno da zero in avanti procedendo da sinistra a destra
(usato nelle CPU SPARC, Motorola e nei server IBM)
- **little endian**, le posizioni dei byte vanno da zero in avanti procedendo da destra a sinistra
(usato nelle CPU Intel)
- RISC V usa *little endian*

il valore 6_{dieci} (memorizzato nella parola a 32 bit di indirizzo 0)
è rappresentato in esadecimale come 00 00 00 06hex

- big endian: 00 00 00 06 (i bit 110 sono nel byte 3)
- little endian: 06 00 00 00 (i bit 110 sono nel byte 0)



big vs little endian – esempio su parole a 32 bit

0	1	2	3
00000000	00000000	00000000	00000110

big endian

0	0	1	2	3
4	4	5	6	7
8	8	9	10	11
12	12	13	14	15

0	3	2	1	0
4	7	6	5	4
8	11	10	9	8
12	15	14	13	12

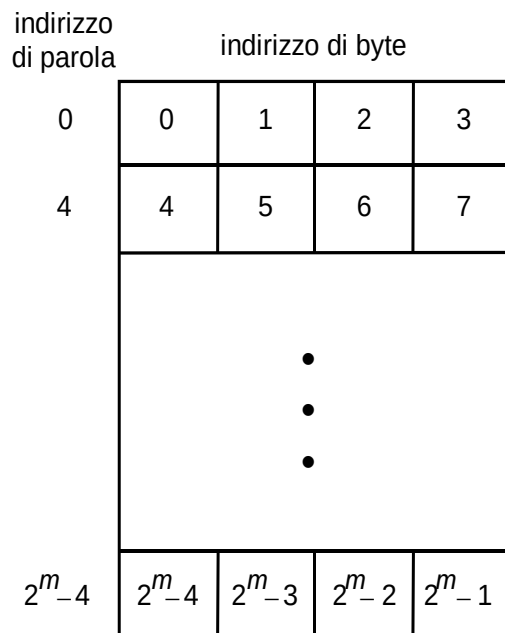
little endian

3	2	1	0
00000000	00000000	00000000	00000110

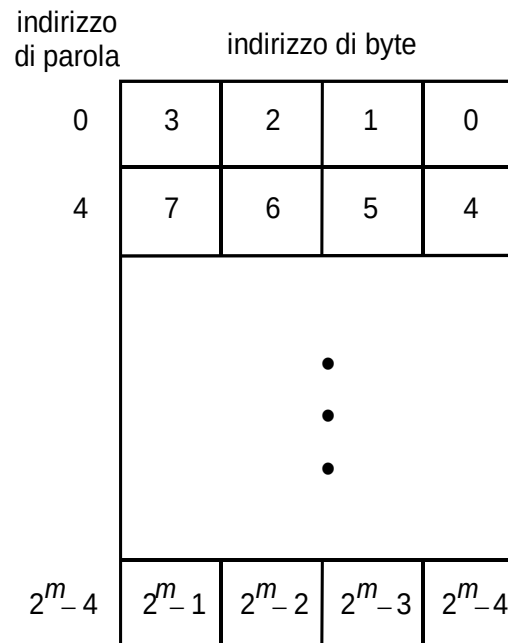


allineamento di parola – esempio su parole a 32 bit

con parole da 32 bit e indirizzamento della memoria per byte, parole consecutive sono collocate a indirizzi multipli di quattro (schema di indirizzamento allineato)



big endian



little endian



add - varianti

addi s1, s2, 100 # add immediate

- addiziona una costante: il valore del secondo addendo è presente nell'istruzione come costante (*immediate*) (qui 100 in base dieci)
- **la costante** deve essere un numero rappresentabile **su 12 bit (in complemento a 2)**
- la costante viene addizionata dopo averla **estesa in segno da 12 bit fino a 64 bit**

RISC V non ha un'istruzione di sottrazione di costante (**subi** non esiste), perché basta usare **addi** con costante cambiata di segno



moltiplicazione di numeri interi

due istruzioni (appartengono alla ISA estesa RV64im):

- `mul rd rs1 rs2` # con segno (**signed**, in CPL2)
- `mulu rd rs1 rs2` # senza segno (**unsigned**)

- i registri *rs1* e *rs2* contengono i *fattori* (gli operandi)
- il registro *rd* contiene il *prodotto* (il risultato)
- considerano solo i 32 bit meno significativi dei registri sorgente *rs1* e *rs2*, mentre il risultato in *rd* può estendersi su 64 bit
- ci sono alcune varianti di *mul* e *mulu* (ma qui non interessano)
- la ISA RV64im ha istruzioni simili anche per la divisione intera (*div* e *divu*) e il calcolo del resto (*rem* e *remu*), con varianti



divisione e resto di interi con risultato intero

divisione: *rs1* dividendo, *rs2* divisore, *rd* quoziente (intero)

- **div** *rd rs1 rs2* # con segno (**signed**, in CPL2)
- **divu** *rd rs1 rs2* # senza segno (**unsigned**)

resto: *rs1* dividendo, *rs2* divisore, *rd* resto

- **rem** *rd rs1 rs2* # con segno (**signed**, in CPL2)
- **remu** *rd rs1 rs2* # senza segno (**unsigned**)

Nota Bene: RISC V non prevede un'istruzione di addizione senza segno (*addu* / *addiu* non esistono); infatti le addizioni con e senza segno, come operazioni aritmetiche, non differiscono tra loro e le istruzioni *add* / *addi* sopperiscono a entrambi gli usi; idem per la sottrazione; **invece le versioni con e senza segno di *mul*, *div* e *rem*, nonché i confronti, sono operazioni differenti e necessitano di istruzioni apposite caratterizzate dal suffisso *u*.**



numeri reali

- si rappresentano in *virgola mobile* (*floating point* – simile alla notazione scientifica), secondo lo standard IEEE 754
- su parole da 32 bit o 64 bit (precisione singola o doppia)
- si caricano in un banco di registri ausiliario: *f0*, *f1*, *f2*, ecc
- apposite istruzioni eseguono le operazioni aritmetiche sui registri di virgola mobile: *fadd*, *fsub*, *fmul*, ecc
- apposite istruzioni permettono di trasferire numeri tra registri in virgola mobile e registri interi, arrotondando se serve
- i metodi di arrotondamento ammissibili sono specificati con esattezza dallo standard IEEE 754
- l'aritmetica standard IEEE 754 è presente su TUTTI i moderni processori, per garantire uniformità nei calcoli scientifici
- alcuni processori hanno estensioni allo standard IEEE 754



istruzioni logiche

operazione	istruzione	significato	esempio
and	and rd, rs1, rs2	and bit a bit tra il contenuto dei due registri	and s1, s2, s3
or	or rd, rs1, rs2	or bit a bit tra il contenuto dei due registri	or s1, s2, s3
xor	xor rd, rs1, rs2	xor bit a bit tra il contenuto dei due registri	xor s1, s2, s3

esistono anche **andi**, **ori** e **xori** che eseguono and / or / xor bit a bit tra un operando e una costante

attenzione: la costante viene prolungata da 12 bit a 64 bit tramite estensione di segno



istruzioni di scorrimento

operazione	istruzione	significato	esempio
shift left logical	slli rd, rs1, cost12	sposta a sinistra il contenuto di <i>rs1</i> del numero di bit specificato, e introduce bit 0 da dx	slli s1, s2, 10
shift right logical	srlr rd, rs1, cost12	sposta a destra il contenuto di <i>rs1</i> del numero di bit specificato, e introduce bit 0 da sx	srlr s1, s2, 10
shift right arith	srai rd, rs1, cost12	sposta a destra il contenuto di <i>rs1</i> del numero di bit specificato, ed estende il segno a sx	srai s1, s2, 10

Nota Bene: **cost12** è costante intera positiva compresa tra 1 e 63

esistono anche le versioni (**sll**, **srl**, **sra**) in cui il terzo argomento è un registro (*rs2*) che specifica il numero di posizioni di scorrimento



array con elementi interi da 64 bit – i

$s3$	A [0]
$s3 + 8$	A [1]
$s3 + 16$	A [2]
	
$s3 + 8 \times i$	A [i]
	

l'elemento **i-esimo** di un array si troverà nella locazione

$$br + 8 \times i$$

dove:

- **br** è il registro base ($s3$ nell'esempio)
- **i** è l'indice dell'elemento dell'array
- il fattore **8** dipende dall'indirizzamento al byte della memoria in RISC V



lettura di un elemento di array

a = A [i]

si assuma che l'indirizzo del primo elemento dell'array (*base address*) sia contenuto nel registro **s3**, **i** sia in **s1**, e **a** sia in **s2**

```
slli t1, s1, 3           # registro t1 contiene  $i \times 8$   
add  t2, t1, s3         # registro t2 contiene  $A + i \times 8$   
                        # ossia l'indirizzo di A [i]  
ld   s2, 0(t2)          # carico in s2 il valore  
                        # dell'elemento dell'array A [i]
```

se poi volessi caricare A [i + 1], basterebbe modificare lo spiazzamento

```
ld   s2, 8(t2)
```



array – esempio di compilazione

codice C: $A[12] = h + A[8]$

si supponga che:

- la variabile **h** sia associata al registro **s2**
- l'indirizzo del primo elemento dell'array (*base address*) sia contenuto nel registro **s3**

codice RISC V:

ld	t0, 64(s3)	# t0 ← memoria[s3 + 64]
add	t0, s2, t0	# t0 ← s2 + t0
sd	t0, 96(s3)	# memoria[s3 + 96] ← t0



array – esempio – i

sia A un array di 100 «parole doppie»

istruzione C: $g = h + A[i]$

si supponga che:

- le variabili **g**, **h**, **i** siano associate rispettivamente ai registri **s1**, **s2** e **s4**
- l'indirizzo del primo elemento dell'array (*base address*) sia contenuto nel registro **s3**



array – esempio – ii

l'elemento **i-esimo** dell'array si trova nella locazione di memoria di indirizzo **$s3 + 8 \times i$**

caricamento dell'indirizzo di **A[i]** nel registro temporaneo **t1**:

```
slli t1, s4, 3      # t1 ← 8 × i
add  t1, t1, s3      # t1 ← indirizzo di A[i]
                        # ossia s3 + 8 × i
```

per trasferire **A[i]** nel registro temporaneo **t0**:

```
ld t0, 0(t1)        # t0 ← A[i]
```

per sommare **h** e **A[i]** e mettere il risultato in **g**:

```
add s1, s2, t0      # g = h + A[i]
```



istruzioni di modifica del flusso di controllo

normalmente, l'esecuzione è sequenziale

- il program counter (PC), indica dove si trova la prossima istruzione da eseguire
- il flusso standard è: esegui l'istruzione all'indirizzo indicato da PC, e avanza all'istruzione successiva ($PC = PC + 4$)
- le «istruzioni per la modifica del flusso» servono a forzare la modifica del PC rispetto al flusso standard sequenziale

possiamo ridirigere l'esecuzione del programma in ogni punto, facendo salti in avanti e indietro

si può modificare il flusso (cioè sovrascrivere il PC) con istruzioni di:

- salto condizionato (**branch**) – subordinato al verificarsi di una condizione
- salto incondizionato (**jump**) – il salto viene sempre eseguito



salти condizionati

i saltи condizionati hanno tutti la forma

branch_condizione rs1, rs2, ind_salto

significato: se confrontando i registri *rs1* e *rs2* la condizione viene soddisfatta, allora salta

branch equal **beq rs1, rs2, ind_salto**
(salta se $rs1 = rs2$)

branch not equal **bne rs1, rs2, ind_salto**
(salta se $rs1 \neq rs2$)

ci sono anche le istruzioni **blt** e **bge** (salta se strettamente minore / se maggiore uguale) e le corrispondenti in aritmetica senza segno (unsigned)

ind_salto: l'indirizzo destinazione di salto è espresso in modo relativo rispetto al PC



salto incondizionato (con collegamento)

le istruzioni di **salto incondizionato**

- specificano l'indirizzo a cui saltare e
- prevedono che il PC dell'istruzione successiva a quella di salto venga salvato in un registro (collegamento/link)

jal	rd, spi20	$rd := pc + 4$ $pc := pc + spi20$	salto incondizionato relativo a PC con salvataggio dell'indirizzo di rientro nel registro <i>rd</i>
jalr	rd, spi12(rs1)	$rd := pc + 4$ $pc := rs1 + spi12$	chiamata a sottoprogramma relativa a registro <i>rs1</i> con salvataggio dell'indirizzo di rientro nel registro <i>rd</i>



salto incondizionato – pseudo-istruzioni

j	spi20	$pc := pc + spi20$	salto incondizionato – salto relativo a PC Equivalente: jal x0, spi20
jr	rs1	$pc := rs1$	salto incondizionato a lunga distanza – salto da registro <i>rs1</i> Equivalente: jalr x0, (rs1)
jal	spi20	$ra := pc + 4$ $pc := pc + spi20$	chiamata a sottoprogramma – salto incondizionato relativo a PC con salvataggio dell'indirizzo di rientro nel registro <i>ra</i> Equivalente: jal ra, spi20
jalr	rs1	$ra := pc + 4$ $pc := rs1$	chiamata a sottoprogramma a lunga distanza – salto incondizionato da registro <i>rs1</i> con salvataggio dell'indirizzo di rientro nel registro <i>ra</i> Equivalente: jalr ra, (rs1)
ret		$pc := ra$	rientro da sottoprogramma – salto da registro <i>ra</i> Equivalente a: jr ra , oppure a: jalr x0, ra, 0



istruzioni di salto condizionato e incondizionato

istruzioni di salto: un nuovo indirizzo viene caricato nel contatore di programma (PC), invece di caricare l'indirizzo che segue quello dell'istruzione di salto stessa secondo l'ordine sequenziale delle istruzioni

istruzioni di **salto condizionato** (*conditional branch*): il salto viene eseguito solo se una certa condizione risulta soddisfatta

esempi: **beq** (*branch on equal*) e **bne** (*branch on not equal*)

```
beq r1, r2, L1    # go to L1 if r1 = r2
```

```
bne r1, r2, L1    # go to L1 if r1 ≠ r2
```

Istruzioni di **salto incondizionato** (*unconditional jump*): il salto viene sempre eseguito

esempi: **j** (*jump*), **jr** (*jump register*) e **jal** (*jump and link*)

```
j    L1           # go to the instruction labelled L1 (PC=L1)
```

```
jr   x31          # go to the address contained in x31
```

```
jal  L1           # go to PC+L1 and save into register ra  
# the address of the next instruction  
# i.e., of the one that follows jal
```



etichette

nelle istruzioni in linguaggio macchina si usano gli indirizzi:

- per un programmatore assembler non è conveniente/possibile calcolare/
- conoscere esplicitamente l'indirizzo numerico di dati/istruzioni
- si usano etichette mnemoniche (label), che si associano a specifici punti
- del programma (istruzioni o dati)
- l'assemblatore poi traduce le etichette nei corrispondenti indirizzi

istruzioni

```
LABEL_NAME:      addi   x4, x5, 10  
                   sub    x4, x4, x6
```

dati (variabili a, b, c, da 8 byte ciascuna)

.data

```
A:  .zero 8  # lascia uno spazio di 8 byte  
B:  .zero 8  # lascia uno spazio di 8 byte  
C:  .zero 8  # lascia uno spazio di 8 byte
```



esempio – *if ... then ... else*

codice C:

```
if (i == j)
    f = g + h
else
    f = g - h
```

si suppone che le variabili **f**, **g**, **h**, **i** e **j** siano associate rispettivamente ai registri **s0**, **s1**, **s2**, **s3** e **s4**

codice RISC V:

```
IF:      bne      s3, s4, ELSE      # go to ELSE if i ≠ j
THEN:    add      s0, s1, s2        # f = g + h (skipped if i ≠ j)
        j        ENDIF            # go to ENDIF
ELSE:    sub      s0, s1, s2        # f = g - h (skipped if i = j)
ENDIF:    .....
```

Nota Bene: le etichette IF e THEN non servono per saltare (e si possono eliminare), ma aiutano a distinguere le varie parti



istruzioni di confronto

spesso è utile confrontare se una variabile è minore di un'altra

slt s1, s2, s3 # set on less than

assegna il valore **1** a **s1** se **s2 < s3**; altrimenti assegna il valore **0**

slti funziona similmente, ma il terzo argomento è una costante (*immediate*) su 12 bit

ATTENZIONE!!

slt / **slti** eseguono i confronti in aritmetica con segno (signed), cioè interpretando i valori numerici come CPL2; se i valori numerici sono degli indirizzi (senza segno – unsigned) bisogna usare **sltu** / **sltiu**

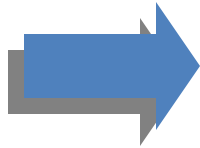


esempio – `slt` e `beq` per realizzare l'equivalente di `blt`

```
if (i < j)
    k = i + j;
else
    k = i - j;
```

```
IF:    blt    s0, s1, ELSE
...

```



```
# s0 e s1 contengono i e j
# s2 contiene k
```

```
IF:    slt    t0, s0, s1
        beq    t0, zero, ELSE
THEN:  add    s2, s0, s1
        j      EXIT
ELSE:  sub    s2, s0, s1
EXIT:
```

istruzioni native assembler RISC V – i

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri
	Sottrazione	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri
	Somma immediata	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola doppia	ld x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola doppia da memoria a registro
	Memorizzazione parola doppia	sd x5, 40(x6)	$\text{Memoria}[x6 + 40] = x5$	Spostamento di una parola doppia da registro a memoria
	Lettura parola	lw x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola da memoria a registro
	Lettura parola senza segno	lwu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una parola senza segno da memoria a registro
	Memorizzazione parola	sw x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una parola da registro a memoria
	Lettura mezza parola	lh x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	lhu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola senza segno da memoria a registro
	Memorizzazione mezza parola	sh x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una mezza parola da registro a memoria
	Lettura byte	lb x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte da memoria a registro
	Lettura byte, senza segno	lbu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte senza segno da memoria a registro
	Memorizzazione byte	sb x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di un byte da registro a memoria
	Lettura di una parola e blocco	lr.d x5, (x6)	$x5 = \text{Memoria}[x6]$	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria
	Memorizzazione condizionata di una parola	sc.d x7, x5, (x6)	$\text{Memoria}[x6] = x5; x7 = 0/1$	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria
	Caricamento costante nella mezza parola superiore	lui x5, 0x12345	$x5 = 0x12345000$	Caricamento di una costante su 20 bit nei <u>20</u> bit più significativi di una parola

aggiungere istruzione *auipc* (vedere prox slide)

istruzioni native assembler RISC V – ii

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Logiche	And	and x5, x6, x7	$x5 = x6 \& x7$	Operandi in tre registri; AND bit a bit
	Or inclusivo	or x5, x6, x8	$x5 = x6 x8$	Operandi in tre registri; OR bit a bit
	Or esclusivo (Xor)	xor x5, x6, x9	$x5 = x6 \wedge x9$	Operandi in tre registri; XOR bit a bit
	And immediato	andi x5, x6, 20	$x5 = x6 \& 20$	AND bit a bit tra un operando in registro e una costante
	Or immediato	ori x5, x6, 20	$x5 = x6 20$	OR bit a bit tra un operando in registro e una costante
	Xor immediato	xori x5, x6, 20	$x5 = x6 \wedge 20$	XOR bit a bit tra un operando in registro e una costante
Scorrimento (shift)	Scorrimento logico a sinistra	sll x5, x6, x7	$x5 = x6 \ll x7$	Scorrimento a sinistra mediante registro
	Scorrimento logico a destra	srl x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra mediante registro
	Scorrimento aritmetico a destra	sra x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra aritmetico mediante registro
	Scorrimento logico a sinistra immediato	slli x5, x6, 3	$x5 = x6 \ll 3$	Scorrimento a sinistra mediante costante
	Scorrimento logico a destra immediato	srlr x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra mediante costante
	Scorrimento aritmetico a destra immediato	srai x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra aritmetico mediante costante
Salti condizionati	Salta se uguale	beq x5, x6, 100	Se $(x5 == x6)$ vai a PC+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne x5, x6, 100	Se $(x5 != x6)$ vai a PC+100	Test di disuguaglianza; salto relativo al PC
	Salta se minore di	blt x5, x6, 100	Se $(x5 < x6)$ vai a PC+100	Comparazione di minoranza; salto relativo al PC
	Salta se maggiore o uguale di	bge x5, x6, 100	Se $(x5 \geq x6)$ vai a PC+100	Comparazione di maggioranza o uguaglianza; salto relativo al PC
	Salta se minore di senza segno	bltu x5, x6, 100	Se $(x5 < x6)$ vai a PC+100	Comparazione di minoranza senza segno; salto relativo al PC
	Salta se maggiore o uguale di senza segno	bgeu x5, x6, 100	Se $(x5 \geq x6)$ vai a PC+100	Comparazione di maggioranza o uguaglianza senza segno; salto relativo al PC
Salti incondizionati	Salta e collega	jal x1, 100	$x1 = PC+4$; vai a PC+100	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	jalr x1, 100(x5)	$x1 = PC+4$; vai a $x5+100$	Ritorno da procedura; chiamata indiretta

istruzioni e formati

Istruzione RISC-V	Nome	Formato	Pseudoistruzione RISC-V	Nome	Istruzione reale
Somma	add	R	Sposta	mv	addi
Sottrazione	sub	R	Carica immediato	li	addi
Add immediato	addi	I	Salta	j	Jal
Lettura parola doppia	ld	I	Carica indirizzo	la	lui + addi
Memorizzazione parola doppia	sd	S			
Lettura parola	lw	I			
Lettura parola senza segno	lwu	I			
Memorizzazione parola	sw	S			
Lettura mezza parola	lh	I			
Lettura mezza parola senza segno	lhu	I			
Memorizzazione mezza parola	sh	S			
Lettura byte	lb	I			
Lettura byte senza segno	lbu	I			
Memorizzazione byte	sb	S			
Lettura riservata	lr.d	R			
Memorizzazione condizionata	sc.d	R			
Lettura dei bit superiori	lui	U			
And	and	R			
Or inclusivo	or	R			
Or esclusivo	xor	R			
And immediato	andi	I			
Or inclusivo immediato	ori	I			
Or esclusivo immediato	xori	I			
Scorrimento logico sinistra	sll	R			
Scorrimento logico destra	srl	R			
Scorrimento aritmetico destra	sra	R			
Scorrimento logico sinistra immediato	slli	I			
Scorrimento logico destra immediato	srl_i	I			
Scorrimento aritmetico destra immediato	srai	I			
Salta se uguale	beq	SB			
Salta se non uguale	bne	SB			
Salta se minore	blt	SB			
Salta se maggiore o uguale	bge	SB			
Salta se minore senza segno	bltu	SB			
Salta se maggiore o uguale senza segno	bgeu	SB			
Jump and link	jal	UJ			
Jump and link a registro	jalr	I			

assembler RISC V

fascicolo assembler RISC V (utilizzabile anche all'esame)

- quadro completo riassuntivo delle istruzioni native e delle pseudo-istruzioni dell'assembler RISC V considerate nel corso
- direttive fondamentali all'assemblatore
- convenzioni sull'uso dei registri

traduzione da C a RISC V: materiale didattico di supporto dettagliato (slide)

- come tradurre da C a RISC V
un modello per generare codice macchina